Client problem and value

Scenario 1: Banking complaint handling

Client challenge

The fictional bank faces a growing complaint backlog, slow searches across customer notes and policy documents, and incomplete handovers to specialist teams. Automation must preserve confidentiality, auditability and human accountability.

Proposed intervention

ComplaintAssist is a staff-facing decision-support prototype, not a customer-facing chatbot or an automated redress engine. It provides suggested triage, policy evidence, missing-field warnings and an editable specialist handover that cannot be approved without a human review.

Potential business value

- Reduce time spent searching notes and reference policies (hypothesis, not yet measured).
- Improve handover completeness by checking category-specific required information.
- Reduce misrouting through consistent classification, while retaining human override.
- Improve traceability through versioned source IDs and recorded employee decisions.

Boundaries and assumptions

The demo uses 30 main synthetic cases, 18 challenge cases, 18 sections from six invented bank policies and local SQLite review records. Policy rules and routing labels are fictional. It does not connect to bank systems, settle complaints, contact customers or issue refunds.

How the prototype works

Technical architecture and tools

Data flow

- 1 Staff selects a fictional complaint and its case notes.
- 2 Input is checked for common PII patterns and suspicious instruction-like text.
- 3 Rules-based triage suggests issue category, urgency and specialist team.
- 4 Lexical retrieval ranks policy sections and exposes versioned citations.
- 5 A structured handover includes chronology, missing fields and next steps.
- 6 Staff inspect evidence, edit the summary and explicitly approve or reject.
- 7 A SQLite audit records generation and human review events.

Tools and trade-offs

- Python + Pydantic: explicit data types and deterministic validation.
- FastAPI + plain HTML browser demo; optional Streamlit interface. Both require a local server.
- SQLite: persistent local audit trail; not tamper-evident or multi-user secure.
- BM25-inspired lexical search: interpretable offline retrieval; no semantic embedding search.
- Optional OpenAI structured triage and evidence-constrained drafting: untested without API access; invalid drafts fall back to templates.

Assessment of AI tool use

Default is offline rules and template drafting. Optional OpenAI mode provides structured triage and source-constrained handover drafting, validated against retrieved citation IDs. The model was not evaluated in this environment; fallback is explicit. Output remains a human-reviewed draft.

Testing, evidence and limitations

Actual local evaluation; not client impact

Executed tests

30 + 18	38.9%	12/12
Template + challenge	Challenge accuracy	Tests passed

- Original template set: 27/30 correct categories (90%); this is a weak generalisation test.
- Phrase-variation challenge set: 7/18 correct categories (38.9%), revealing brittle rules.
- Challenge retrieval: 8/18 labelled policy sections appeared in the top four (44.4%).
- Original set: 30/30 cases required review; 30/30 citations had valid IDs, not proven support.
- 12 tests passed, including API routes, confirmation checks and safe AI fallback.

Limitations of these measurements

- Both sets are small and authored for this project; the second set deliberately varies wording.
- No real employee handling-time measurements, cost savings or financial ROI have been established.
- Optional OpenAI triage and handover generation are untested without an API key.
- Recall@4 tests retrieving a labelled reference, not claim-level citation support.

Next evaluation stage

Before piloting: collect independently authored blind cases, improve routing/retrieval, verify claim support, assess PII leakage and time trained handlers on manual versus assisted workflows.

Governance and human controls

Responsible AI and operational safeguards

Controls implemented in the demonstration

- Synthetic data only: no genuine customer information is required.
- Regex redaction for common email, telephone and payment-card patterns before optional LLM triage.
- A limited pattern-based warning for instruction-like input, with human inspection required.
- Pydantic validation of category, team, priority and handover structure.
- Every handover is a draft until an employee explicitly confirms review and records approval.
- Review confirmation, changes and decisions are recorded in local SQLite; approvals lock further edits.

Residual risk and required production controls

- Regex redaction and pattern matching are incomplete; data-loss-prevention checks need broader coverage.
- Policy retrieval can return weak or irrelevant passages; verify evidence applicability manually.
- No authentication, role-based access, immutable audit or enterprise secrets management.
- No live banking integration, approved policy governance, model monitoring or incident response.
- Business rules are hypothetical; compliance requirements need review by bank policy and legal teams.

Decision rights

The system proposes information. A bank employee verifies facts, priorities, evidence, omissions and final handover content. Specialist decisions, compensation, fraud investigations and account actions remain entirely outside the prototype.

Workforce impact and next steps

Adoption, assumptions and recommendation

How the workflow changes

- Today (illustrative): a handler reads complaint notes, searches multiple documents, prepares a summary and sends it to a specialist.
- Proposed: the handler starts with an AI-assisted triage and draft, checks citations, corrects omissions, and approves the handover.
- Specialists receive more structured inputs; managers monitor handover quality and override patterns.
- Staff require training in verification, data protection, handling uncertainty and escalating incorrect AI recommendations.

Assumptions and trade-offs

- Assumes structured case notes and approved policy sources can be made available in a future bank environment.
- Deterministic baseline maximises reproducibility, but misses language variation; LLM triage may improve coverage with additional complexity.
- Local SQLite keeps the demo easy to open, but limits multi-user security and deployment readiness.
- Mandatory human review adds effort but makes accountability explicit.

Practical next steps

- Evaluate against independently written, held-out examples and difficult edge cases.
- Trial with complaint handlers to measure actual time, completeness and override rates.
- Harden privacy, authentication, permissions, audit integrity and source governance.
- Conduct formal business-risk, accessibility and compliance reviews before considering a pilot.

Recommendation

Treat ComplaintAssist as a testable proof of concept for staff decision support. Proceed toward a controlled pilot only if independent evaluation and staff trials demonstrate measurable benefits without unacceptable safety, privacy or operational trade-offs.